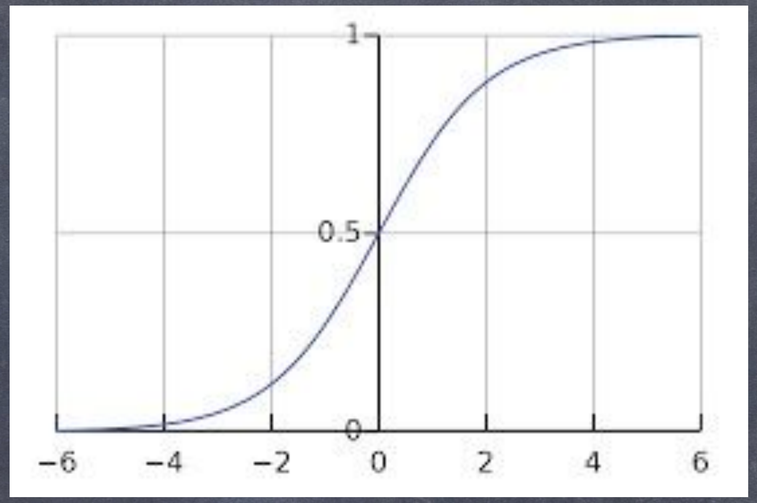
Neural Network Contd..

Bias in Neural Network

- In simple words, neural network bias can be defined as the constant which is added to the product of features and weights
- It is used to offset the result
- It helps the models to shift the activation function towards the positive or negative side

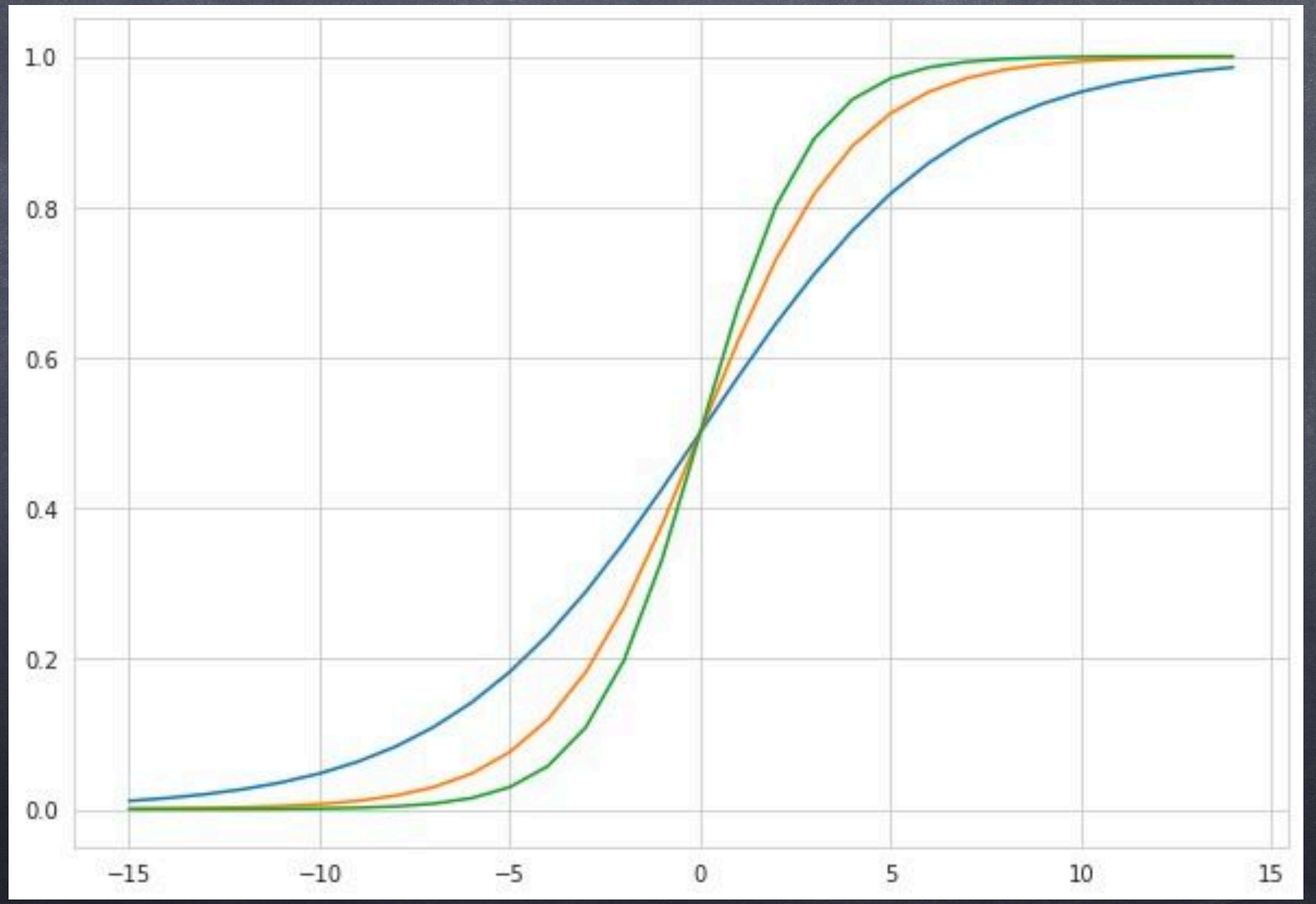
Let us understand it by using sigmoid function



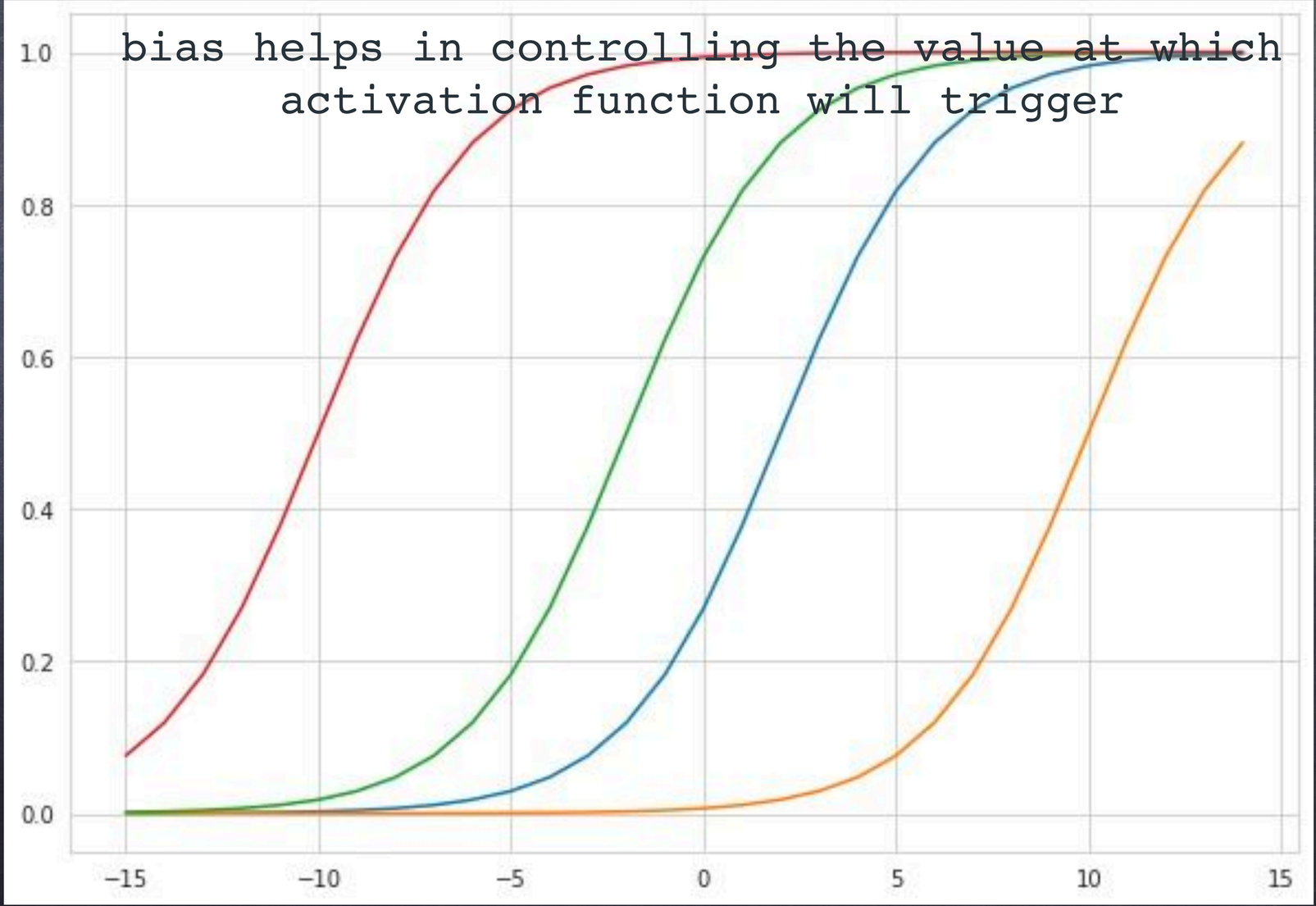
$$sigmoid\ function = \frac{1}{1 + e^{-x}}$$

$$sigmoid\ function = \frac{1}{1 + e^{-(w*x+b)}}$$

Fix b = 0 and vary w

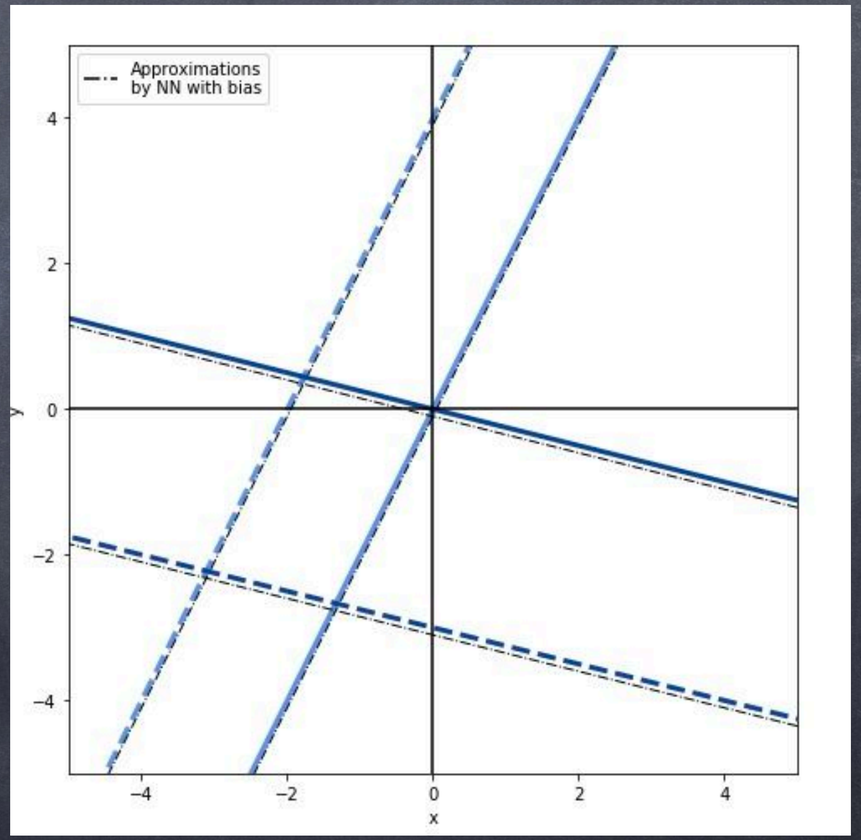


Fix w and vary b



One bias per layer vs one bias per node

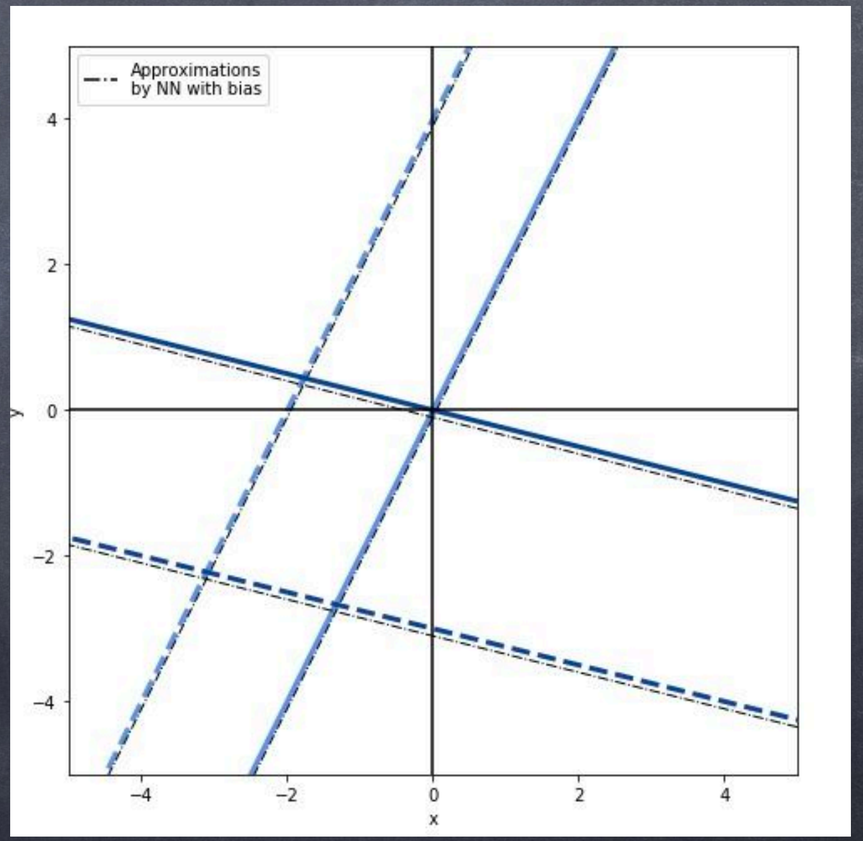
- Consider a feature vector X =[x1, x2, ….,xn] and their associated systematic errors (b1+ b2+ ….bn) for n inputs
- A single layer neural network computing a function
- Y =f(X,W) + (b1+ b2+ ….bn)



https://www.turing.com/kb/necessity-of-bias-in-neural-networks

One bias per layer vs one bias per node

- Consider a feature vector X =[x1, x2, ….,xn] and their associated systematic errors (b1+ b2+ ….bn) for n inputs
- A single layer neural network computing a function
- Y =f(X,W) + b)



https://www.turing.com/kb/necessity-of-bias-in-neural-networks

One bias per layer vs one bias per node

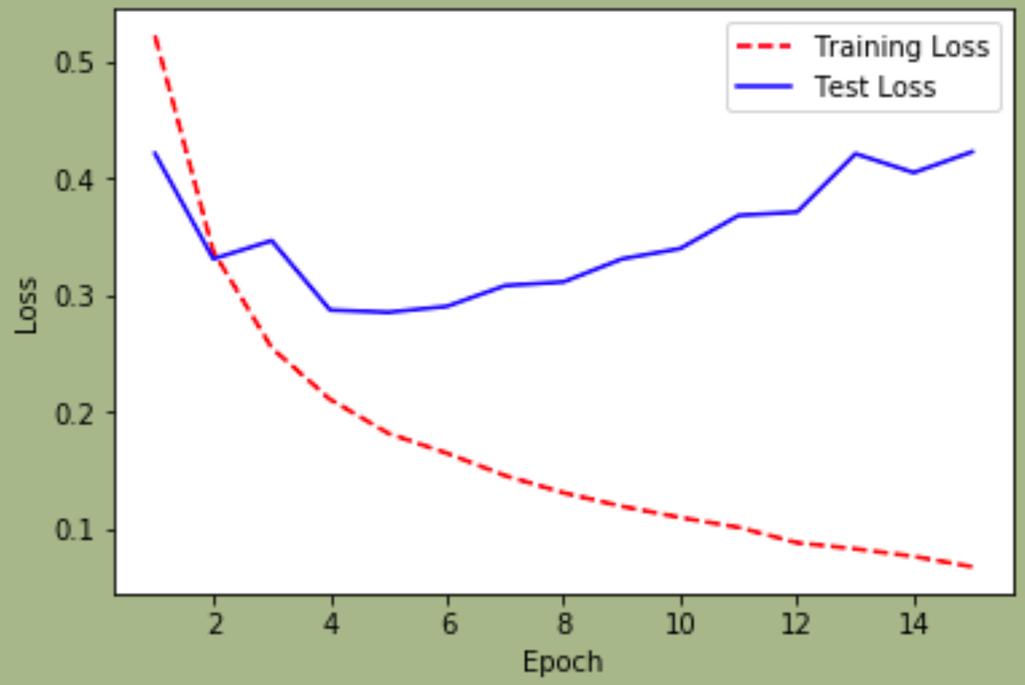
- Expand this to the network with multiple layers

$$y_k(\mathbf{x}, \mathbf{w}) = \sigma\left(\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right)$$

$$f(X) = \sigma_n(W_n\sigma_{n-1}(...(W_2\sigma_1(W_1X + b_1) + b_2) + ...) + b_n)$$

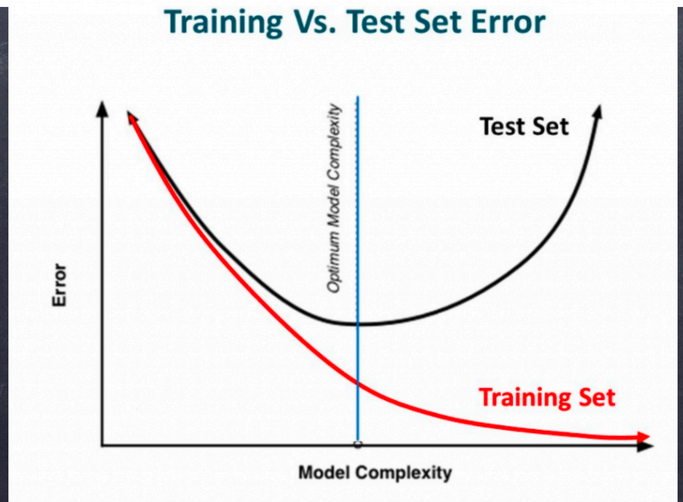
https://www.turing.com/kb/necessity-of-bias-in-neural-networks

No of epochs and training error

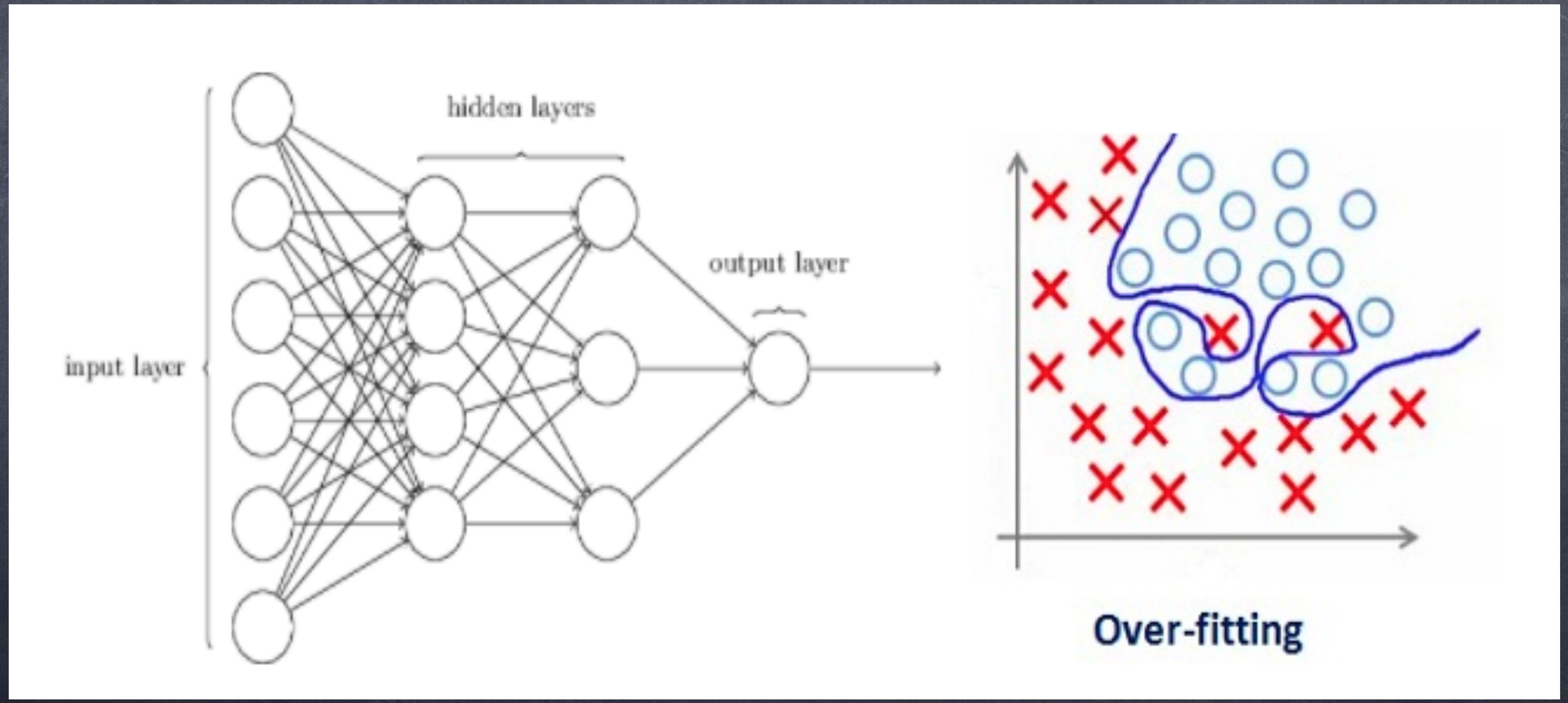


What is regularization

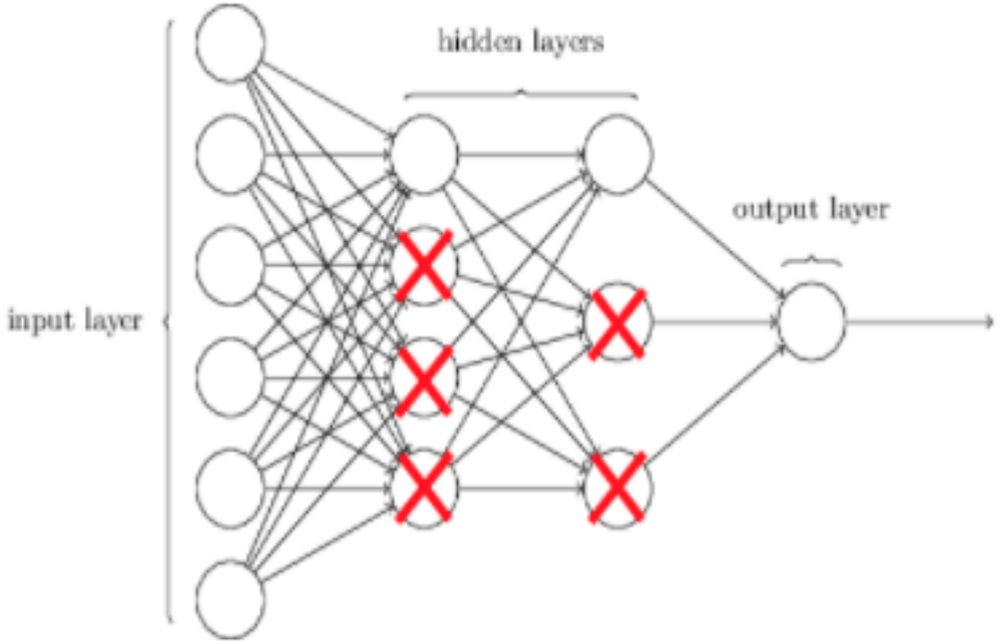


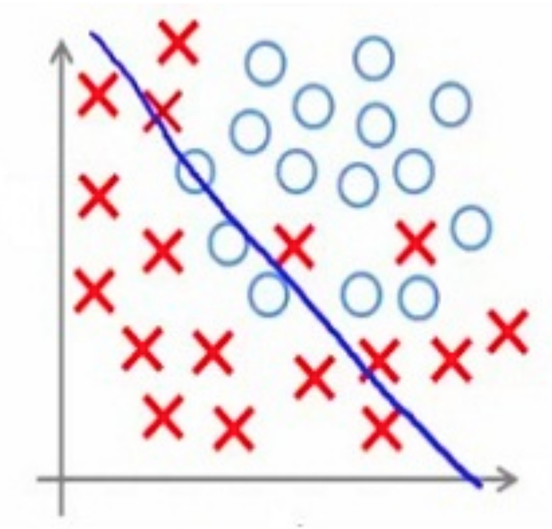


Overfitting

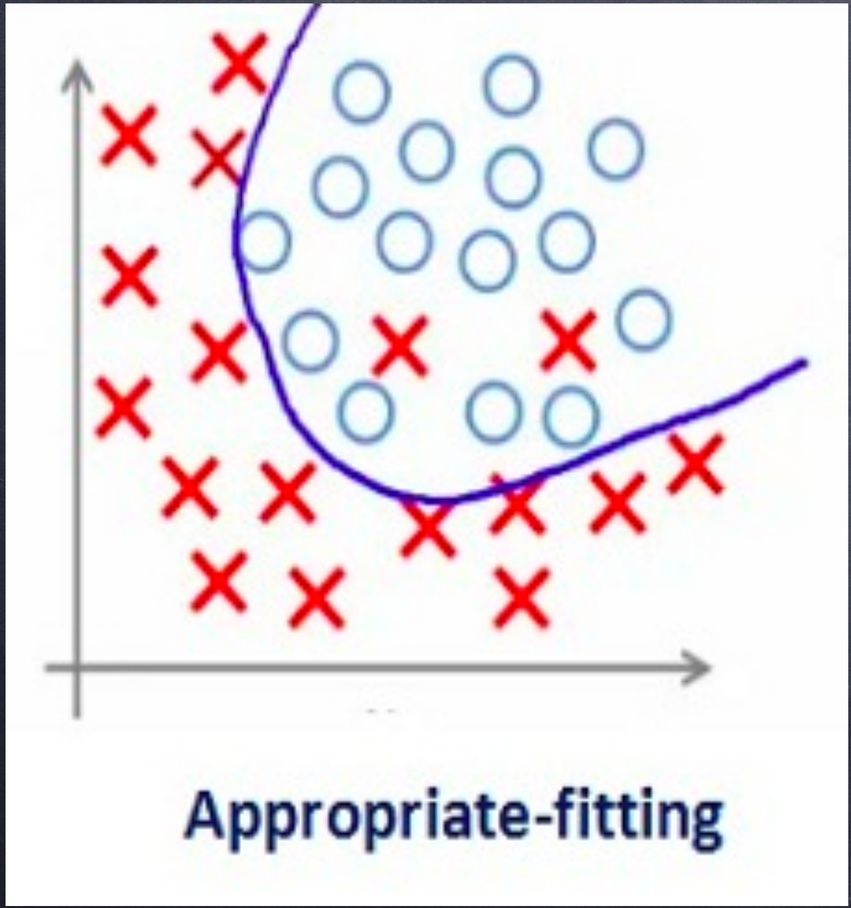


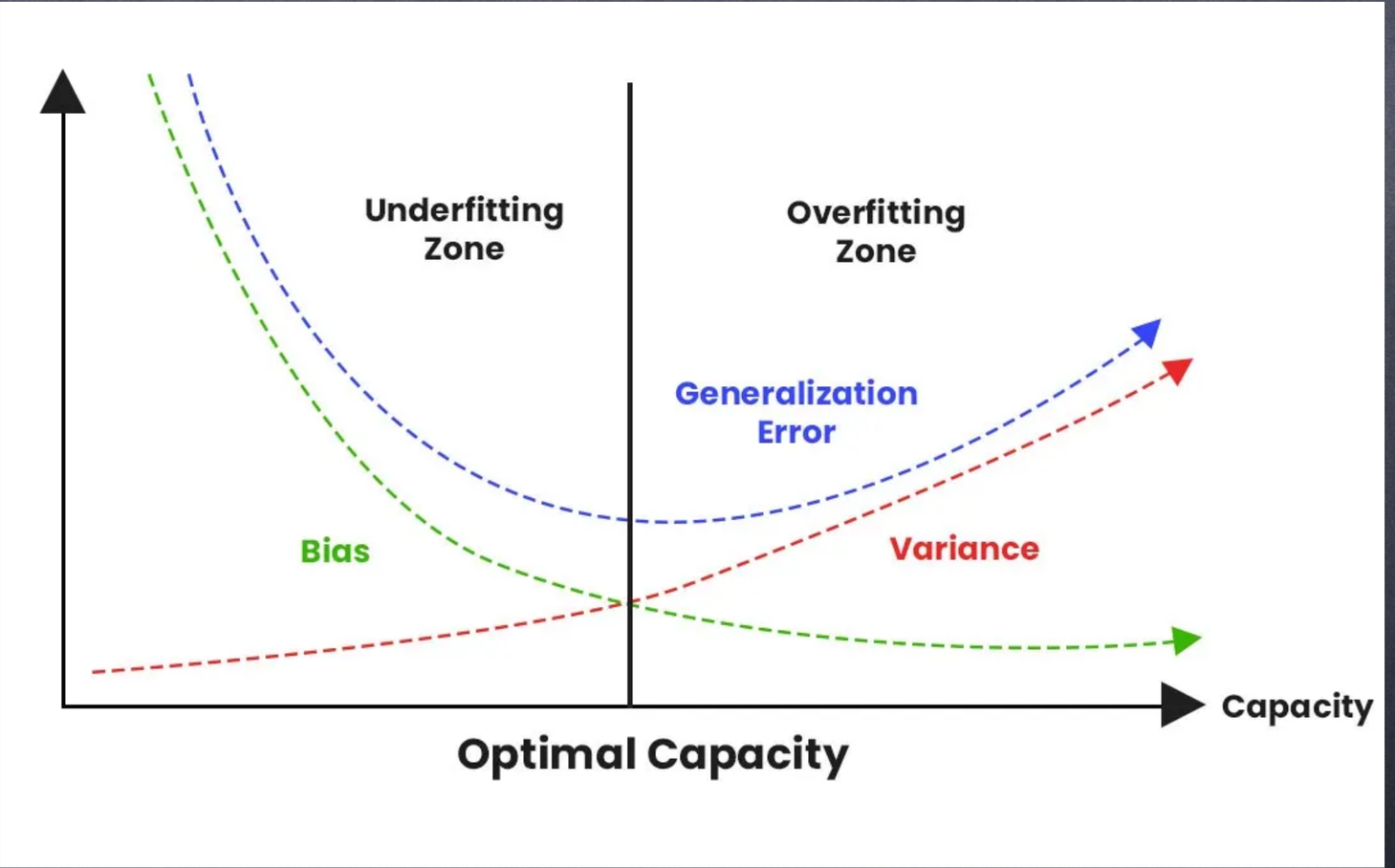
Under-fitting

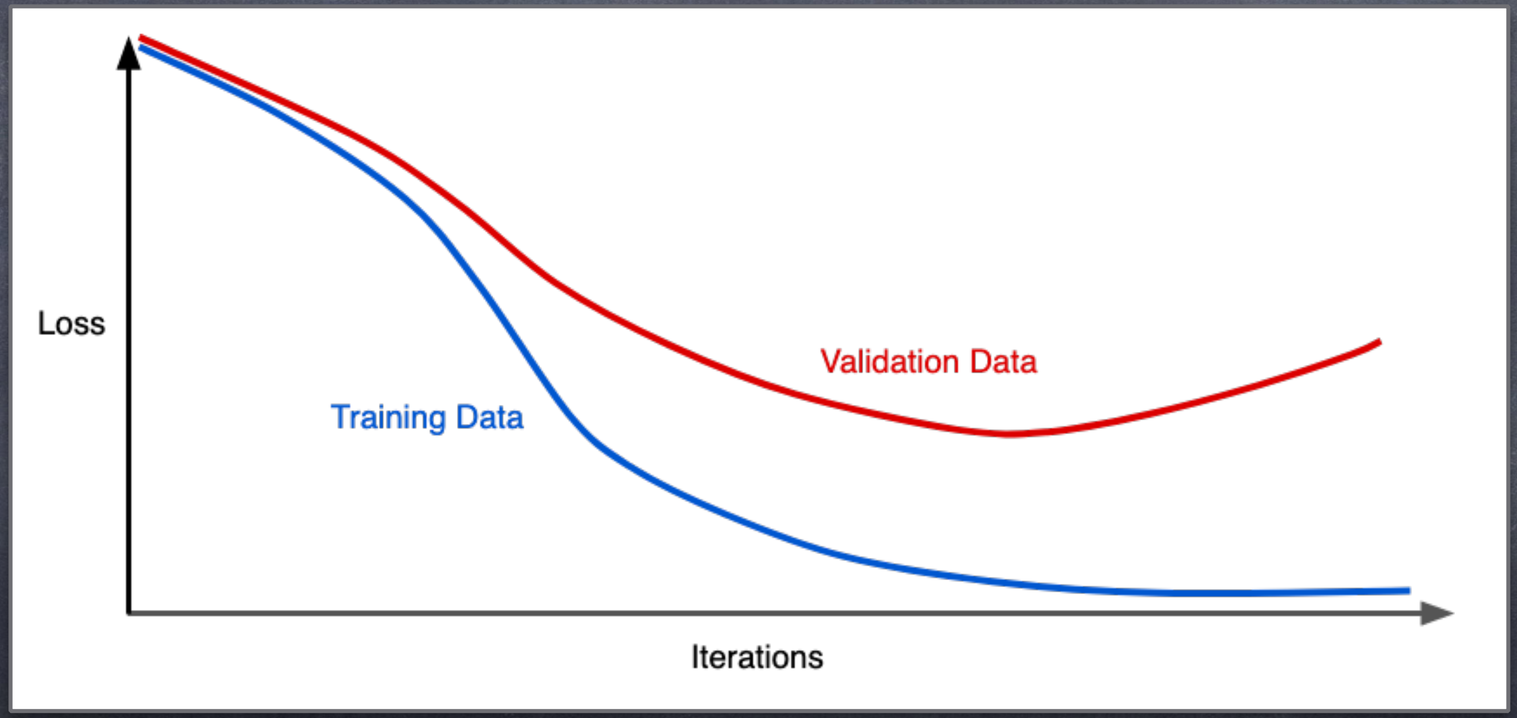




Under-fitting


Appropriate-fitting


Underfitting Zone
Overfitting Zone
Generalization Error
Bias
Variance
Capacity
Optimal Capacity



Early stopping?

Should we allow all possible weights?

Any preferences?

instead of simply aiming to minimize loss (empirical risk minimization): minimize(Loss(DatalModel))

we'll now minimize loss+complexity, which is called **structural risk minimization**:

$$\text{minimize}(\text{Loss}(\text{DatalModel}) + \text{complexity}(\text{Model}))$$

two terms:

the **loss term**, which measures how well the model fits the data,

the **regularization term**, which measures model complexity

- Generally, we don't want huge weights
- If weights are large, a small change in a feature can result in a large change in the prediction
- Also gives too much weight to any one feature
- Might also prefer weights of 0 for features that aren't useful

$$\text{argmin}_{w,b} \sum_{i=1}^{n} loss(yy') + \lambda\ regularizer(w,b)$$

sum of the weights

$$r(w,b) = \sum_{w_j} |w_j|$$

sum of the squared weights

$$r(w,b) = \sqrt{\sum_{w_j} |w_j|^2}$$

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i(w \cdot x_i + b)) - \eta \lambda w_j$$

learning rate

direction to update

constant: how far from wrong

regularization

L1 Regularizer

Least Absolute Shrinkage and Selection Operator

$$\text{argmin}_{w,b} \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b)) + \|w\|$$

$$\frac{d}{dw_j} objective = \frac{d}{dw_j} \sum_{i=1}^{n} \exp(-y_i(w \cdot x_i + b)) + \lambda \|w\|$$

$$= -\sum_{i=1}^{n} y_i x_{ij} \exp(-y_i(w \cdot x_i + b)) + \lambda sign(w_j)$$

L1 Regularization

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i(w \cdot x_i + b)) - \eta\lambda sign(w_j)$$

learning rate

direction to update

constant: how far from wrong

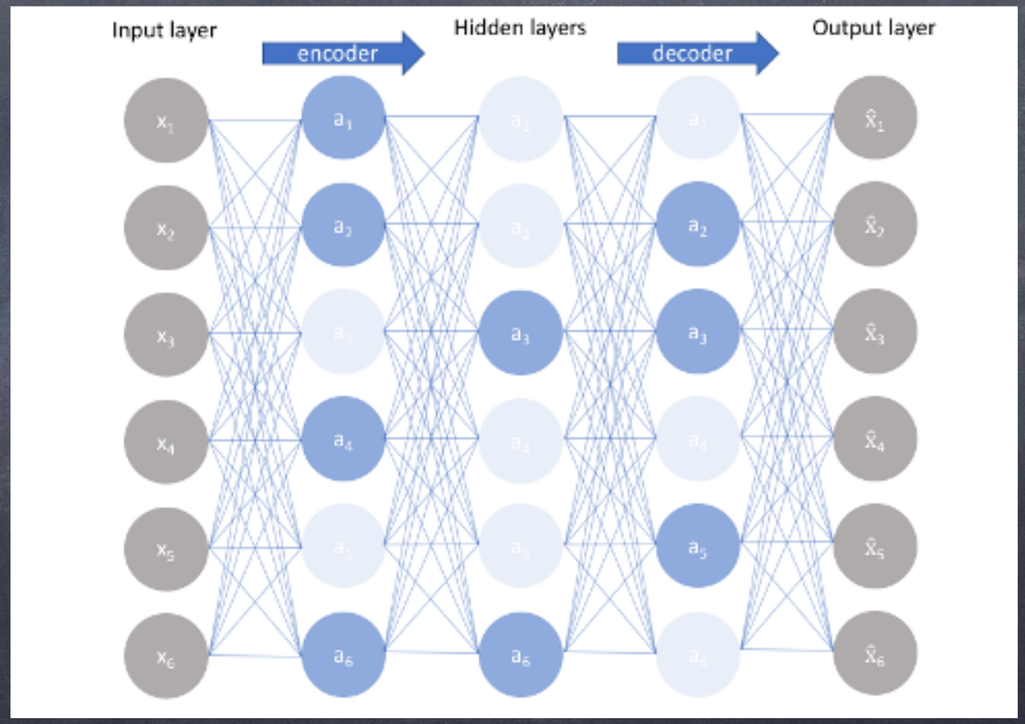
regularization

If w_i is positive, reduces by a constant

If w_i is negative, increases by a constant

In other words ...

- Construct a loss function to penalize activations
- L1 Regularization penalizes the absolute value of the vector of activations



L2 Regularizer
Ridge regression (Thikonov regularization)

$$\text{L2-Regularized-Cost}_{\mathbf{w},\mathbf{b}} = \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}(y^{[i]}, \hat{y}^{[i]}) + \frac{\lambda}{n}\sum_{l=1}^{L}||\mathbf{w}^{(l)}||_F^2$$

sum over layers

Regular gradient descent update:

$$w_{i,j} := w_{i,j} - \eta\frac{\partial\mathcal{L}}{\partial w_{i,j}}$$

Gradient descent update with L2 regularization:

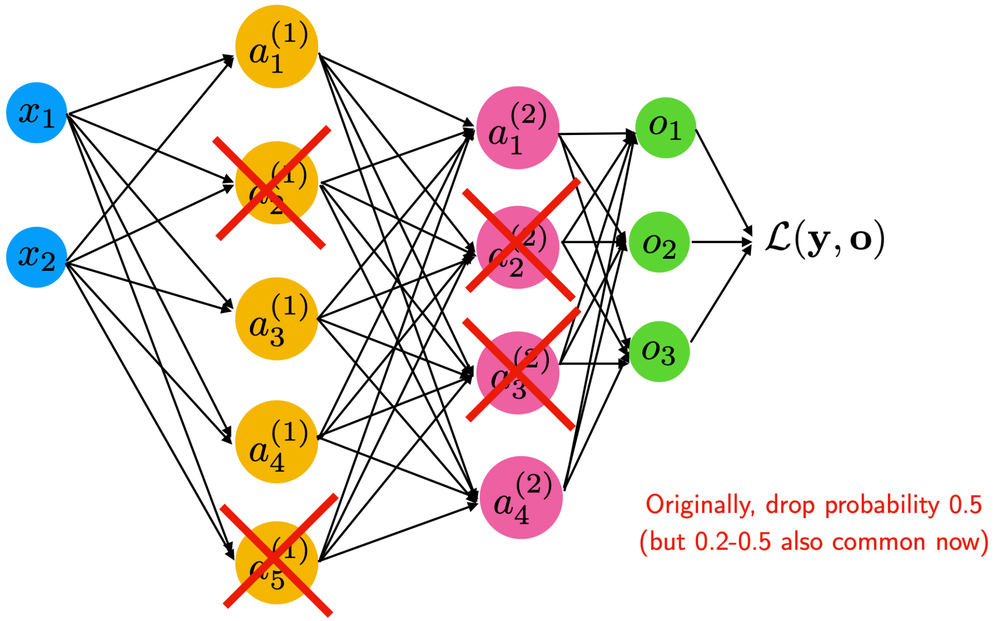
$$w_{i,j} := w_{i,j} - \eta\left(\frac{\partial\mathcal{L}}{\partial w_{i,j}} + \frac{2\lambda}{n}w_{i,j}\right)$$

- L2
  - Default choice for deep learning: L2 is smoother mathematically, making it easier and more stable for modern gradient-based optimizers to handle.
  - Distributed importance: Use when you expect most input features or hidden units to contribute a little bit to the final prediction.
  - Preventing large weights: Use to curb runaway weight values caused by noisy data or multicollinearity.
- L1
  - Feature selection: Use when you have a massive, high-dimensional input dataset and want the network to discover and drop irrelevant or redundant inputs automatically.
  - Model compression / Sparsity: Use when you want to zero out inactive neurons or connections to save memory and speed up inference.

Best of both the worlds -> L1+L2 (Elastic Net)

- L2
  - Default choice for deep learning: L2 is smoother mathematically, making it easier and more stable for modern gradient-based optimizers to handle.
  - Distributed importance: Use when you expect most input features or hidden units to contribute a little bit to the final prediction.
  - Preventing large weights: Use to curb runaway weight values caused by noisy data or multicollinearity.
- L1
  - Feature selection: Use when you have a massive, high-dimensional input dataset and want the network to discover and drop irrelevant or redundant inputs automatically.
  - Model compression / Sparsity: Use when you want to zero out inactive neurons or connections to save memory and speed up inference.

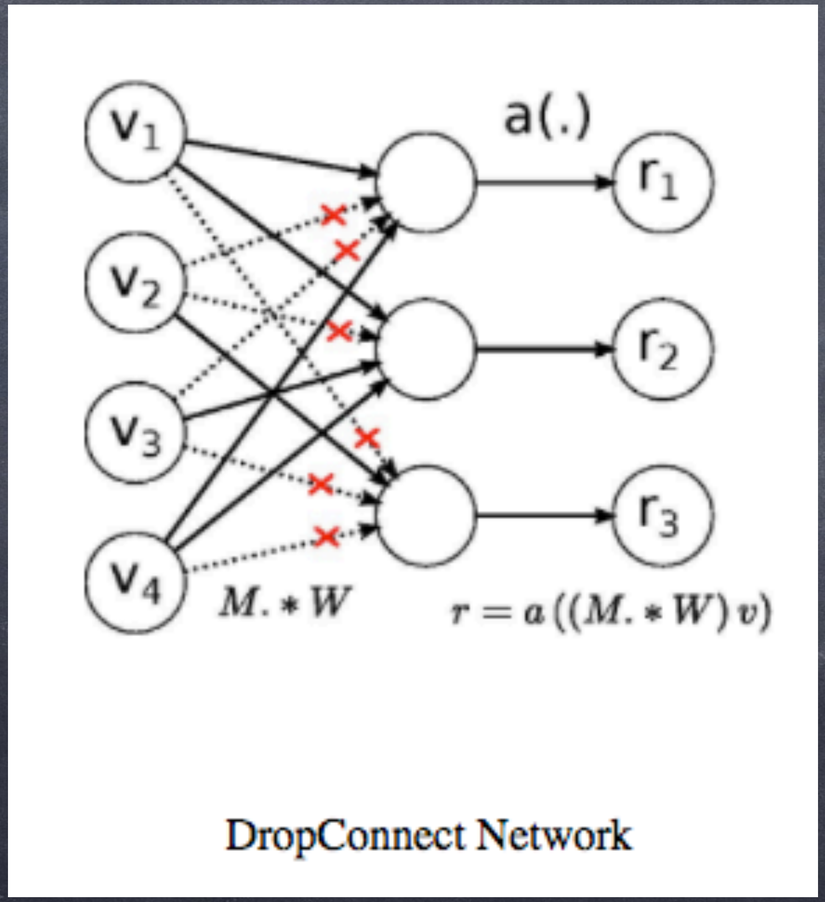
Dropout as regularization



- Randomly drops nodes at every iteration of the training.
- Trained model behave as an ensemble of multiple models.

Srivastava, Nitish, et al. "Dropout: a simple way to prevent neural networks from overfitting." *The Journal of Machine Learning Research* 15.1 (2014): 1929-1958.

DropConnet



DropConnect Network